Features

Feature List
Comparison to Other Database Engines
H2 in Use
Connection Modes
Database URL Overview
Connecting to an Embedded (Local) Database
In-Memory Databases
Database Files Encryption
Database File Locking
Opening a Database Only if it Already Exists
Closing a Database
Ignore Unknown Settings
Changing Other Settings when Opening a Connection
Custom File Access Mode
Multiple Connections
Database File Layout
Logging and Recovery
Compatibility
Auto-Reconnect
Automatic Mixed Mode
Page Size
Using the Trace Options
Using Other Logging APIs
Read Only Databases
Read Only Databases in Zip or Jar File
Computed Columns / Function Based Index
Multi-Dimensional Indexes
User-Defined Functions and Stored Procedures
Pluggable or User-Defined Tables
Triggers
Compacting a Database
Cache Settings

Feature List

Main Features

- Very fast database engine
- Open source
- Written in Java
- Supports standard SQL, JDBC API
- Embedded and Server mode, Clustering support
- Strong security features
- The PostgreSQL ODBC driver can be used
- Multi version concurrency

Additional Features

- Disk based or in-memory databases and tables, read-only database support, temporary tables
- Transaction support (read committed), 2-phase-commit
- Multiple connections, table level locking
- Cost based optimizer, using a genetic algorithm for complex queries, zero-administration
- Scrollable and updatable result set support, large result set, external result sorting, functions can return a result set
- Encrypted database (AES), SHA-256 password encryption, encryption functions, SSL

SQL Support

- Support for multiple schemas, information schema

- Referential integrity / foreign key constraints with cascade, check constraints
- Inner and outer joins, subqueries, read only views and inline views
- Triggers and Java functions / stored procedures
- Many built-in functions, including XML and lossless data compression
- Wide range of data types including large objects (BLOB/CLOB) and arrays
- Sequence and autoincrement columns, computed columns (can be used for function based indexes)
- ORDER BY, GROUP BY, HAVING, UNION, LIMIT, TOP
- Collation support, including support for the ICU4J library
- Support for users and roles
- Compatibility modes for IBM DB2, Apache Derby, HSQLDB, MS SQL Server, MySQL, Oracle, and PostgreSQL.

Security Features

- Includes a solution for the SQL injection problem
- User password authentication uses SHA-256 and salt
- For server mode connections, user passwords are never transmitted in plain text over the network (even wl using insecure connections; this only applies to the TCP server and not to the H2 Console however; it also doesn't apply if you set the password in the database URL)
- All database files (including script files that can be used to backup data) can be encrypted using the AES-' encryption algorithm
- The remote JDBC driver supports TCP/IP connections over TLS
- The built-in web server supports connections over TLS
- Passwords can be sent to the database using char arrays instead of Strings

Other Features and Tools

- Small footprint (smaller than 1.5 MB), low memory requirements
- Multiple index types (b-tree, tree, hash)
- Support for multi-dimensional indexes
- CSV (comma separated values) file support
- Support for linked tables, and a built-in virtual 'range' table
- Supports the EXPLAIN PLAN statement; sophisticated trace options
- Database closing can be delayed or disabled to improve the performance
- Web-based Console application (translated to many languages) with autocomplete
- The database can generate SQL script files
- Contains a recovery tool that can dump the contents of the database
- Support for variables (for example to calculate running totals)
- Automatic re-compilation of prepared statements
- Uses a small number of database files
- Uses a checksum for each record and log entry for data integrity
- Well tested (high code coverage, randomized stress tests)

Comparison to Other Database Engines

This comparison is based on H2 1.3, Apache Derby version 10.8, HSQLDB 2.2, MySQL 5.5, PostgreSQL 9.0.

Feature	H2	Derby	HSQLDB	MySQL	PostgreSQL
Pure Java	Yes	Yes	Yes	No	No
Embedded Mode (Java)	Yes	Yes	Yes	No	No
In-Memory Mode	Yes	Yes	Yes	No	No
Explain Plan	Yes	Yes *12	Yes	Yes	Yes
Built-in Clustering / Replication	Yes	Yes	No	Yes	Yes
Encrypted Database	Yes	Yes *10	Yes *10	No	No
Linked Tables	Yes	No	Partially *1	Partially *2	Yes
ODBC Driver	Yes	No	No	Yes	Yes
Fulltext Search	Yes	Yes	No	Yes	Yes

Domains (User-Defined Types)	Yes	No	Yes	Yes	Yes
Files per Database	Few	Many	Few	Many	Many
Row Level Locking	Yes *9	Yes	Yes *9	Yes	Yes
Multi Version Concurrency	Yes	No	Yes	Yes	Yes
Multi-Threaded Processing	No *11	Yes	Yes	Yes	Yes
Role Based Security	Yes	Yes *3	Yes	Yes	Yes
Updatable Result Sets	Yes	Yes *7	Yes	Yes	Yes
Sequences	Yes	Yes	Yes	No	Yes
Limit and Offset	Yes	Yes *13	Yes	Yes	Yes
Window Functions	No *15	No *15	No	No	Yes
Temporary Tables	Yes	Yes *4	Yes	Yes	Yes
Information Schema	Yes	No *8	Yes	Yes	Yes
Computed Columns	Yes	Yes	Yes	Yes	Yes *6
Case Insensitive Columns	Yes	Yes *14	Yes	Yes	Yes *6
Custom Aggregate Functions	Yes	No	Yes	No	Yes
CLOB/BLOB Compression	Yes	No	No	No	Yes
Footprint (jar/dll size)	~1.5 MB *5	~3 MB	~1.5 MB	~4 MB	~6 MB

*1 HSQLDB supports text tables.
*2 MySQL supports linked MySQL tables under the name 'federated tables'.
*3 Derby support for roles based security and password checking as an option.
*4 Derby only supports global temporary tables.
*5 The default H2 jar file contains debug information, jar files for other databases do not.
*6 PostgreSQL supports functional indexes.
*7 Derby only supports updatable result sets if the query is not sorted.
*8 Derby doesn't support standard compliant information schema tables.
*9 When using MVCC (multi version concurrency).
*10 Derby and HSQLDB don't hide data patterns well.
*11 The MULTI_THREADED option is not enabled by default, and with version 1.3.x not supported when using MVCC.
*12 Derby doesn't support the EXPLAIN statement, but it supports runtime statistics and retrieving statemen execution plans.
*13 Derby doesn't support the syntax LIMIT .. [OFFSET ..] , however it supports FETCH FIRST .. ROW[S] ONLY .
*14 Using collations. *15 Derby and H2 support ROW_NUMBER() OVER() .

DaffodilDb and One$Db

It looks like the development of this database has stopped. The last release was February 2006.

McKoi

It looks like the development of this database has stopped. The last release was August 2004.

H2 in Use

For a list of applications that work with or use H2, see: Links.

Connection Modes

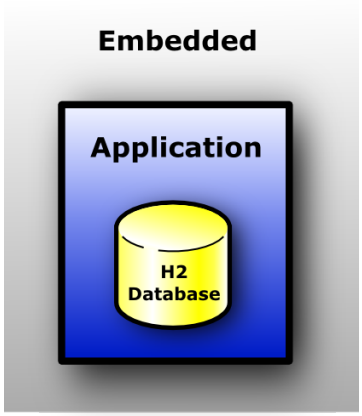
The following connection modes are supported:

- Embedded mode (local connections using JDBC)
- Server mode (remote connections using JDBC or ODBC over TCP/IP)
- Mixed mode (local and remote connections at the same time)

Embedded Mode

In embedded mode, an application opens a database from within the same JVM using JDBC. This is the faste and easiest connection mode. The disadvantage is that a database may only be open in one virtual machine (

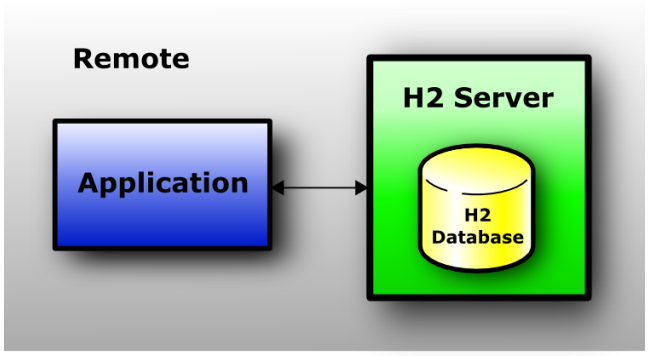
class loader) at any time. As in all modes, both persistent and in-memory databases are supported. There is n limit on the number of database open concurrently, or on the number of open connections.



Server Mode

When using the server mode (sometimes called remote mode or client/server mode), an application opens a database remotely using the JDBC or ODBC API. A server needs to be started within the same or another virt machine, or on another computer. Many applications can connect to the same database at the same time, by connecting to this server. Internally, the server process opens the database(s) in embedded mode.

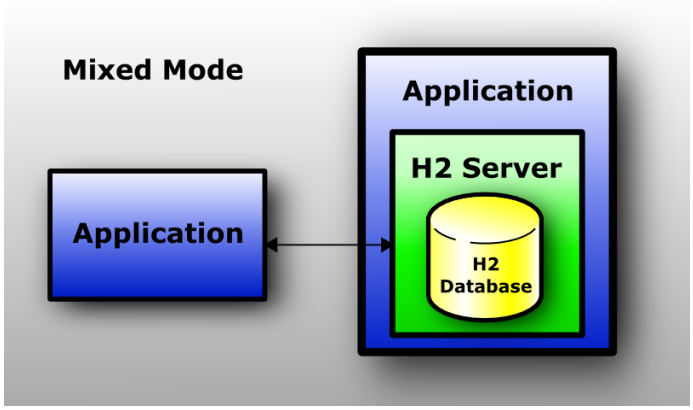
The server mode is slower than the embedded mode, because all data is transferred over TCP/IP. As in all modes, both persistent and in-memory databases are supported. There is no limit on the number of database open concurrently per server, or on the number of open connections.



Mixed Mode

The mixed mode is a combination of the embedded and the server mode. The first application that connects tc database does that in embedded mode, but also starts a server so that other applications (running in different processes or virtual machines) can concurrently access the same data. The local connections are as fast as i the database is used in just the embedded mode, while the remote connections are a bit slower.

The server can be started and stopped from within the application (using the server API), or automatically (automatic mixed mode). When using the automatic mixed mode, all clients that want to connect to the databa (no matter if it's an local or remote connection) can do so using the exact same database URL.



Database URL Overview

This database supports multiple connection modes and connection settings. This is achieved using different database URLs. Settings in the URLs are not case sensitive.

Topic	URL Format and Examples
Embedded (local) connection	jdbc:h2:[file:][<path>]<databaseName> jdbc:h2:~/test jdbc:h2:file:/data/sample jdbc:h2:file:C:/data/sample (Windows only)
In-memory (private)	jdbc:h2:mem:
In-memory (named)	jdbc:h2:mem:<databaseName> jdbc:h2:mem:test_mem
Server mode (remote connections) using TCP/IP	jdbc:h2:tcp://<server>[:<port>]/[<path>]<databaseName> jdbc:h2:tcp://localhost/~/test jdbc:h2:tcp://dbserv:8084/~/sample jdbc:h2:tcp://localhost/mem:test
Server mode (remote connections) using TLS	jdbc:h2:ssl://<server>[:<port>]/<databaseName> jdbc:h2:ssl://localhost:8085/~/sample;
Using encrypted files	jdbc:h2:<url>;CIPHER=AES jdbc:h2:ssl://localhost/~/test;CIPHER=AES jdbc:h2:file:~/secure;CIPHER=AES
File locking methods	jdbc:h2:<url>;FILE_LOCK={FILE|SOCKET|NO} jdbc:h2:file:~/private;CIPHER=AES;FILE_LOCK=SOCKET
Only open if it already exists	jdbc:h2:<url>;IFEXISTS=TRUE jdbc:h2:file:~/sample;IFEXISTS=TRUE
Don't close the database when the VM exits	jdbc:h2:<url>;DB_CLOSE_ON_EXIT=FALSE
Execute SQL on connection	jdbc:h2:<url>;INIT=RUNSCRIPT FROM '~/create.sql' jdbc:h2:file:~/sample;INIT=RUNSCRIPT FROM '~/create.sql'\;RUNSCRIPT FROM '~/populate.sql'
User name and/or password	jdbc:h2:<url>[;USER=<username>][;PASSWORD=<value>] jdbc:h2:file:~/sample;USER=sa;PASSWORD=123
Debug trace settings	jdbc:h2:<url>;TRACE_LEVEL_FILE=<level 0..3> jdbc:h2:file:~/sample;TRACE_LEVEL_FILE=3
Ignore unknown settings	jdbc:h2:<url>;IGNORE_UNKNOWN_SETTINGS=TRUE
Custom file access mode	jdbc:h2:<url>;ACCESS_MODE_DATA=rws
Database in a zip file	jdbc:h2:zip:<zipFileName>!/<databaseName> jdbc:h2:zip:~/db.zip!/test
Compatibility mode	jdbc:h2:<url>;MODE=<databaseType> jdbc:h2:~/test;MODE=MYSQL
Auto-reconnect	jdbc:h2:<url>;AUTO_RECONNECT=TRUE jdbc:h2:tcp://localhost/~/test;AUTO_RECONNECT=TRUE
Automatic mixed mode	jdbc:h2:<url>;AUTO_SERVER=TRUE jdbc:h2:~/test;AUTO_SERVER=TRUE
Page size	jdbc:h2:<url>;PAGE_SIZE=512
Changing other settings	jdbc:h2:<url>;<setting>=<value>[;<setting>=<value>...] jdbc:h2:file:~/sample;TRACE_LEVEL_SYSTEM_OUT=3

Connecting to an Embedded (Local) Database

The database URL for connecting to a local database is jdbc:h2:[file:][<path>]<databaseName> . The prefix file: is optional. If no or only a relative path is used, then the current working directory is used as a starting point. The case sensitivity of the path and database name depend on the operating system, however it is recommended to use lowercase letters only. The database name must be at least three characters long (a

limitation of File.createTempFile). The database name must not contain a semicolon. To point to the user hor directory, use ~/ , as in: jdbc:h2:~/test .

In-Memory Databases

For certain use cases (for example: rapid prototyping, testing, high performance operations, read-only databas it may not be required to persist data, or persist changes to the data. This database supports the in-memory mode, where the data is not persisted.

In some cases, only one connection to a in-memory database is required. This means the database to be ope is private. In this case, the database URL is jdbc:h2:mem: Opening two connections within the same virtual machine means opening two different (private) databases.

Sometimes multiple connections to the same in-memory database are required. In this case, the database UR must include a name. Example: jdbc:h2:mem:db1 . Accessing the same database using this URL only works within the same virtual machine and class loader environment.

To access an in-memory database from another process or from another computer, you need to start a TCP server in the same process as the in-memory database was created. The other processes then need to acces the database over TCP/IP or TLS, using a database URL such as: jdbc:h2:tcp://localhost/mem:db1 .

By default, closing the last connection to a database closes the database. For an in-memory database, this means the content is lost. To keep the database open, add ;DB_CLOSE_DELAY=-1 to the database URL. To keep the content of an in-memory database as long as the virtual machine is alive, use jdbc:h2:mem:test;DB_CLOSE_DELAY=-1 .

Database Files Encryption

The database files can be encrypted. Three encryption algorithms are supported:

- "AES" - also known as Rijndael, only AES-128 is implemented.
- "XTEA" - the 32 round version.
- "FOG" - pseudo-encryption only useful for hiding data from a text editor.

To use file encryption, you need to specify the encryption algorithm (the 'cipher') and the file password (in addit to the user password) when connecting to the database.

Creating a New Database with File Encryption

By default, a new database is automatically created if it does not exist yet. To create an encrypted database, connect to it as it would already exist.

Connecting to an Encrypted Database

The encryption algorithm is set in the database URL, and the file password is specified in the password field, before the user password. A single space separates the file password and the user password; the file passwor itself may not contain spaces. File passwords and user passwords are case sensitive. Here is an example to connect to a password-encrypted database:

```
Class.forName("org.h2.Driver");
String url = "jdbc:h2:~/test;CIPHER=AES";
String user = "sa";
String pwds = "filepwd userpwd";
conn = DriverManager.
    getConnection(url, user, pwds);
```

Encrypting or Decrypting a Database

To encrypt an existing database, use the ChangeFileEncryption tool. This tool can also decrypt an encrypted database, or change the file encryption key. The tool is available from within the H2 Console in the tools sectic or you can run it from the command line. The following command line will encrypt the database test in the us home directory with the file password filepwd and the encryption algorithm AES:

```
java -cp h2*.jar org.h2.tools.ChangeFileEncryption -dir ~ -db test -cipher AES -encrypt filepwd
```

Database File Locking

Whenever a database is opened, a lock file is created to signal other processes that the database is in use. If database is closed, or if the process that opened the database terminates, this lock file is deleted.

The following file locking methods are implemented:

- The default method is FILE and uses a watchdog thread to protect the database file. The watchdog reads the lock file each second.
- The second method is SOCKET and opens a server socket. The socket method does not require reading lock file every second. The socket method should only be used if the database files are only accessed by (and always the same) computer.
- The third method is FS . This will use native file locking using FileChannel.lock .
- It is also possible to open the database without file locking; in this case it is up to the application to protect the database files. Failing to do so will result in a corrupted database. Using the method NO forces the database to not create a lock file at all. Please note that this is unsafe as another process is able to open t same database, possibly leading to data corruption.

To open the database with a different file locking method, use the parameter FILE_LOCK . The following code opens the database with the 'socket' locking method:

```
String url = "jdbc:h2:~/test;FILE_LOCK=SOCKET";
```

For more information about the algorithms, see Advanced / File Locking Protocols.

Opening a Database Only if it Already Exists

By default, when an application calls DriverManager.getConnection(url, ...) and the database specified in the URL does not yet exist, a new (empty) database is created. In some situations, it is better to restrict creating databases, and only allow to open existing databases. To do this, add ;IFEXISTS=TRUE to the database UR In this case, if the database does not already exist, an exception is thrown when trying to connect. The connection only succeeds when the database already exists. The complete URL may look like this:

```
String url = "jdbc:h2:/data/sample;IFEXISTS=TRUE";
```

Closing a Database

Delayed Database Closing

Usually, a database is closed when the last connection to it is closed. In some situations this slows down the application, for example when it is not possible to keep at least one connection open. The automatic closing o database can be delayed or disabled with the SQL statement SET DB_CLOSE_DELAY <seconds> . The parameter <seconds> specifies the number of seconds to keep a database open after the last connection to it was closed. The following statement will keep a database open for 10 seconds after the last connection was closed:

```
SET DB_CLOSE_DELAY 10
```

The value -1 means the database is not closed automatically. The value 0 is the default and means the databa is closed when the last connection is closed. This setting is persistent and can be set by an administrator only is possible to set the value in the database URL: jdbc:h2:~/test;DB_CLOSE_DELAY=10 .

Don't Close a Database when the VM Exits

By default, a database is closed when the last connection is closed. However, if it is never closed, the databa is closed when the virtual machine exits normally, using a shutdown hook. In some situations, the database should not be closed in this case, for example because the database is still used at virtual machine shutdown store the shutdown process in the database for example). For those cases, the automatic closing of the datab can be disabled in the database URL. The first connection (the one that is opening the database) needs to set option in the database URL (it is not possible to change the setting afterwards). The database URL to disable database closing on exit is:

```
String url = "jdbc:h2:~/test;DB_CLOSE_ON_EXIT=FALSE";
```

Execute SQL on Connection

Sometimes, particularly for in-memory databases, it is useful to be able to execute DDL or DML commands automatically when a client connects to a database. This functionality is enabled via the INIT property. Note th multiple commands may be passed to INIT, but the semicolon delimiter must be escaped, as in the example below.

```
String url = "jdbc:h2:mem:test;INIT=runscript from '~/create.sql'\\;runscript from '~/init.sql'";
```

Please note the double backslash is only required in a Java or properties file. In a GUI, or in an XML file, only one backslash is required:

```
<property name="url" value=
"jdbc:h2:mem:test;INIT=create schema if not exists test\;runscript from '~/sql/init.sql'"
/>
```

Backslashes within the init script (for example within a runscript statement, to specify the folder names in Windows) need to be escaped as well (using a second backslash). It might be simpler to avoid backslashes in folder names for this reason; use forward slashes instead.

Ignore Unknown Settings

Some applications (for example OpenOffice.org Base) pass some additional parameters when connecting to th database. Why those parameters are passed is unknown. The parameters `PREFERDOSLIKELINEENDS` anc `IGNOREDRIVERPRIVILEGES` are such examples; they are simply ignored to improve the compatibility with OpenOffice.org. If an application passes other parameters when connecting to the database, usually the database throws an exception saying the parameter is not supported. It is possible to ignored such parameters adding `;IGNORE_UNKNOWN_SETTINGS=TRUE` to the database URL.

Changing Other Settings when Opening a Connection

In addition to the settings already described, other database settings can be passed in the database URL. Adc `;setting=value` at the end of a database URL is the same as executing the statement `SET setting value` just after connecting. For a list of supported settings, see SQL Grammar or the DbSettings javadoc.

Custom File Access Mode

Usually, the database opens the database file with the access mode `rw`, meaning read-write (except for read only databases, where the mode `r` is used). To open a database in read-only mode if the database file is not read-only, use `ACCESS_MODE_DATA=r`. Also supported are `rws` and `rwd`. This setting must be specified the database URL:

```
String url = "jdbc:h2:~/test;ACCESS_MODE_DATA=rws";
```

For more information see Durability Problems. On many operating systems the access mode `rws` does not guarantee that the data is written to the disk.

Multiple Connections

Opening Multiple Databases at the Same Time

An application can open multiple databases at the same time, including multiple connections to the same database. The number of open database is only limited by the memory available.

Multiple Connections to the Same Database: Client/Server

If you want to access the same database at the same time from different processes or computers, you need t use the client / server mode. In this case, one process acts as the server, and the other processes (that could reside on other computers as well) connect to the server via TCP/IP (or TLS over TCP/IP for improved securit

Multithreading Support

This database is multithreading-safe. If an application is multi-threaded, it does not need to worry about synchronizing access to the database. An application should normally use one connection per thread. This database synchronizes access to the same connection, but other databases may not do this. To get higher concurrency, you need to use multiple connections.

By default, requests to the same database are synchronized. That means an application can use multiple thre that access the same database at the same time, however if one thread executes a long running query, the ot threads need to wait. To enable concurrent database usage, see the setting MULTI_THREADED .

Locking, Lock-Timeout, Deadlocks

Please note MVCC is enabled in version 1.4.x by default, when using the MVStore. In this case, table level locking is not used. If multi-version concurrency is not used, the database uses table level locks to give each connection a consistent state of the data. There are two kinds of locks: read locks (shared locks) and write loc (exclusive locks). All locks are released when the transaction commits or rolls back. When using the default transaction isolation level 'read committed', read locks are already released after each statement.

If a connection wants to reads from a table, and there is no write lock on the table, then a read lock is added t the table. If there is a write lock, then this connection waits for the other connection to release the lock. If a connection cannot get a lock for a specified time, then a lock timeout exception is thrown.

Usually, SELECT statements will generate read locks. This includes subqueries. Statements that modify dat use write locks. It is also possible to lock a table exclusively without modifying data, using the statement SELECT ... FOR UPDATE . The statements COMMIT and ROLLBACK releases all open locks. The commands SAVEPOINT and ROLLBACK TO SAVEPOINT don't affect locks. The locks are also released when the autocommit mode changes, and for connections with autocommit set to true (this is the default), loc are released after each statement. The following statements generate locks:

Type of Lock	SQL Statement
Read	SELECT * FROM TEST; CALL SELECT MAX(ID) FROM TEST; SCRIPT;
Write	SELECT * FROM TEST WHERE 1=0 FOR UPDATE;
Write	INSERT INTO TEST VALUES(1, 'Hello'); INSERT INTO TEST SELECT * FROM TEST; UPDATE TEST SET NAME='Hi'; DELETE FROM TEST;
Write	ALTER TABLE TEST ...; CREATE INDEX ... ON TEST ...; DROP INDEX ...;

The number of seconds until a lock timeout exception is thrown can be set separately for each connection usi the SQL command SET LOCK_TIMEOUT <milliseconds> . The initial lock timeout (that is the timeout used f new connections) can be set using the SQL command SET DEFAULT_LOCK_TIMEOUT <milliseconds> . Th default lock timeout is persistent.

Avoiding Deadlocks

To avoid deadlocks, ensure that all transactions lock the tables in the same order (for example in alphabetical order), and avoid upgrading read locks to write locks. Both can be achieved using explicitly locking tables usin SELECT ... FOR UPDATE .

Database File Layout

The following files are created for persistent databases:

File Name	Description	Number of Files
test.h2.db	Database file. Contains the transaction log, indexes, and data for all tables. Format: <database>.h2.db	1 per database
test.lock.db	Database lock file. Automatically (re-)created while the database is in use. Format: <database>.lock.db	1 per database (only if in use
test.trace.db	Trace file (if the trace option is enabled). Contains trace information. Format: <database>.trace.db Renamed to <database>.trace.db.old is too big.	0 or 1 per database
test.lobs.db/*	Directory containing one file for each BLOB or CLOB value larger than a certain size.	1 per large object

	Format: <id>.t<tableId>.lob.db	
test.123.temp.db	Temporary file. Contains a temporary blob or a large result set. Format: <database>.<id>.temp.db	1 per object

Moving and Renaming Database Files

Database name and location are not stored inside the database files.

While a database is closed, the files can be moved to another directory, and they can be renamed as well (as long as all files of the same database start with the same name and the respective extensions are unchanged)

As there is no platform specific data in the files, they can be moved to other operating systems without proble

Backup

When the database is closed, it is possible to backup the database files.

To backup data while the database is running, the SQL commands SCRIPT and BACKUP can be used.

Logging and Recovery

Whenever data is modified in the database and those changes are committed, the changes are written to the transaction log (except for in-memory objects). The changes to the main data area itself are usually written lat on, to optimize disk access. If there is a power failure, the main data area is not up-to-date, but because the changes are in the transaction log, the next time the database is opened, the changes are re-applied automatically.

Compatibility

All database engines behave a little bit different. Where possible, H2 supports the ANSI SQL standard, and tri to be compatible to other databases. There are still a few differences however:

In MySQL text columns are case insensitive by default, while in H2 they are case sensitive. However H2 supports case insensitive columns as well. To create the tables with case insensitive texts, append IGNORECASE=TRUE to the database URL (example: jdbc:h2:~/test;IGNORECASE=TRUE).

Compatibility Modes

For certain features, this database can emulate the behavior of specific databases. However, only a small sub of the differences between databases are implemented in this way. Here is the list of currently supported mode and the differences to the regular mode:

DB2 Compatibility Mode

To use the IBM DB2 mode, use the database URL jdbc:h2:~/test;MODE=DB2 or the SQL statement SET MODE DB2 .

- For aliased columns, ResultSetMetaData.getColumnName() returns the alias name and getTableName() returns null .
- Support for the syntax [OFFSET .. ROW] [FETCH ... ONLY] as an alternative for LIMIT .. OFFSET .
- Concatenating NULL with another value results in the other value.
- Support the pseudo-table SYSIBM.SYSDUMMY1.

Derby Compatibility Mode

To use the Apache Derby mode, use the database URL jdbc:h2:~/test;MODE=Derby or the SQL statement SET MODE Derby .

- For aliased columns, ResultSetMetaData.getColumnName() returns the alias name and getTableName() returns null .
- For unique indexes, NULL is distinct. That means only one row with NULL in one of the columns is allow
- Concatenating NULL with another value results in the other value.
- Support the pseudo-table SYSIBM.SYSDUMMY1.

HSQLDB Compatibility Mode

To use the HSQLDB mode, use the database URL jdbc:h2:~/test;MODE=HSQLDB or the SQL statement SE MODE HSQLDB .

- For aliased columns, ResultSetMetaData.getColumnName() returns the alias name and getTableName() returns null .
- When converting the scale of decimal data, the number is only converted if the new scale is smaller than tl current scale. Usually, the scale is converted and 0s are added if required.
- For unique indexes, NULL is distinct. That means only one row with NULL in one of the columns is allow
- Text can be concatenated using '+'.

MS SQL Server Compatibility Mode

To use the MS SQL Server mode, use the database URL jdbc:h2:~/test;MODE=MSSQLServer or the SQL statement SET MODE MSSQLServer .

- For aliased columns, ResultSetMetaData.getColumnName() returns the alias name and getTableName() returns null .
- Identifiers may be quoted using square brackets as in [Test] .
- For unique indexes, NULL is distinct. That means only one row with NULL in one of the columns is allow
- Concatenating NULL with another value results in the other value.
- Text can be concatenated using '+'.

MySQL Compatibility Mode

To use the MySQL mode, use the database URL jdbc:h2:~/test;MODE=MySQL or the SQL statement SET MODE MySQL .

- When inserting data, if a column is defined to be NOT NULL and NULL is inserted, then a 0 (or empty string, or the current timestamp for timestamp columns) value is used. Usually, this operation is not allowe and an exception is thrown.
- Creating indexes in the CREATE TABLE statement is allowed using INDEX(..) or KEY(..) . Example: create table test(id int primary key, name varchar(255), key idx_name(name));
- Meta data calls return identifiers in lower case.
- When converting a floating point number to an integer, the fractional digits are not truncated, but the value i rounded.
- Concatenating NULL with another value results in the other value.

Text comparison in MySQL is case insensitive by default, while in H2 it is case sensitive (as in most other databases). H2 does support case insensitive text comparison, but it needs to be set separately, using SET IGNORECASE TRUE . This affects comparison using =, LIKE, REGEXP .

Oracle Compatibility Mode

To use the Oracle mode, use the database URL jdbc:h2:~/test;MODE=Oracle or the SQL statement SET MODE Oracle .

- For aliased columns, ResultSetMetaData.getColumnName() returns the alias name and getTableName() returns null .
- When using unique indexes, multiple rows with NULL in all columns are allowed, however it is not allowed have multiple rows with the same values otherwise.
- Concatenating NULL with another value results in the other value.
- Empty strings are treated like NULL values.

PostgreSQL Compatibility Mode

To use the PostgreSQL mode, use the database URL jdbc:h2:~/test;MODE=PostgreSQL or the SQL stateme SET MODE PostgreSQL .

- For aliased columns, ResultSetMetaData.getColumnName() returns the alias name and getTableName() returns null .
- When converting a floating point number to an integer, the fractional digits are not be truncated, but the val is rounded.
- The system columns CTID and OID are supported.
- LOG(x) is base 10 in this mode.

Auto-Reconnect

The auto-reconnect feature causes the JDBC driver to reconnect to the database if the connection is lost. The automatic re-connect only occurs when auto-commit is enabled; if auto-commit is disabled, an exception is thrown. To enable this mode, append ;AUTO_RECONNECT=TRUE to the database URL.

Re-connecting will open a new session. After an automatic re-connect, variables and local temporary tables definitions (excluding data) are re-created. The contents of the system table INFORMATION_SCHEMA.SESSION_STATE contains all client side state that is re-created.

If another connection uses the database in exclusive mode (enabled using SET EXCLUSIVE 1 or SET EXCLUSIVE 2), then this connection will try to re-connect until the exclusive mode ends.

Automatic Mixed Mode

Multiple processes can access the same database without having to start the server manually. To do that, append ;AUTO_SERVER=TRUE to the database URL. You can use the same database URL independent of whether the database is already open or not. This feature doesn't work with in-memory databases. Example database URL:

```
jdbc:h2:/data/test;AUTO_SERVER=TRUE
```

Use the same URL for all connections to this database. Internally, when using this mode, the first connection t the database is made in embedded mode, and additionally a server is started internally (as a daemon thread). the database is already open in another process, the server mode is used automatically. The IP address and p of the server are stored in the file .lock.db , that's why in-memory databases can't be supported.

The application that opens the first connection to the database uses the embedded mode, which is faster than the server mode. Therefore the main application should open the database first if possible. The first connectio automatically starts a server on a random port. This server allows remote connections, however only to this database (to ensure that, the client reads .lock.db file and sends the the random key that is stored there to th server). When the first connection is closed, the server stops. If other (remote) connections are still open, one them will then start a server (auto-reconnect is enabled automatically).

All processes need to have access to the database files. If the first connection is closed (the connection that started the server), open transactions of other connections will be rolled back (this may not be a problem if you don't disable autocommit). Explicit client/server connections (using jdbc:h2:tcp:// or ssl://) are not supportec This mode is not supported for in-memory databases.

Here is an example how to use this mode. Application 1 and 2 are not necessarily started on the same compu but they need to have access to the database files. Application 1 and 2 are typically two different processes (however they could run within the same process).

```
// Application 1:
DriverManager.getConnection("jdbc:h2:/data/test;AUTO_SERVER=TRUE");

// Application 2:
DriverManager.getConnection("jdbc:h2:/data/test;AUTO_SERVER=TRUE");
```

When using this feature, by default the server uses any free TCP port. The port can be set manually using AUTO_SERVER_PORT=9090 .

Page Size

The page size for new databases is 2 KB (2048), unless the page size is set explicitly in the database URL us PAGE_SIZE= when the database is created. The page size of existing databases can not be changed, so th property needs to be set when the database is created.

Using the Trace Options

To find problems in an application, it is sometimes good to see what database operations where executed. Thi database offers the following trace features:

- Trace to System.out and/or to a file
- Support for trace levels OFF, ERROR, INFO, DEBUG
- The maximum size of the trace file can be set
- It is possible to generate Java source code from the trace file
- Trace can be enabled at runtime by manually creating a file

Trace Options

The simplest way to enable the trace option is setting it in the database URL. There are two settings, one for System.out (TRACE_LEVEL_SYSTEM_OUT) tracing, and one for file tracing (TRACE_LEVEL_FILE). The trace levels are 0 for OFF , 1 for ERROR (the default), 2 for INFO , and 3 for DEBUG . A database URL wit both levels set to DEBUG is:

```
jdbc:h2:~/test;TRACE_LEVEL_FILE=3;TRACE_LEVEL_SYSTEM_OUT=3
```

The trace level can be changed at runtime by executing the SQL command SET TRACE_LEVEL_SYSTEM_OUT level (for System.out tracing) or SET TRACE_LEVEL_FILE level (for file tracing). Example:

```
SET TRACE_LEVEL_SYSTEM_OUT 3
```

Setting the Maximum Size of the Trace File

When using a high trace level, the trace file can get very big quickly. The default size limit is 16 MB, if the trac file exceeds this limit, it is renamed to .old and a new file is created. If another such file exists, it is deleted. limit the size to a certain number of megabytes, use SET TRACE_MAX_FILE_SIZE mb . Example:

```
SET TRACE_MAX_FILE_SIZE 1
```

Java Code Generation

When setting the trace level to INFO or DEBUG , Java source code is generated as well. This simplifies reproducing problems. The trace file looks like this:

```
...
12-20 20:58:09 jdbc[0]:
/**/dbMeta3.getURL();
12-20 20:58:09 jdbc[0]:
/**/dbMeta3.getTables(null, "", null, new String[]{"TABLE", "VIEW"});
...
```

To filter the Java source code, use the ConvertTraceFile tool as follows:

```
java -cp h2*.jar org.h2.tools.ConvertTraceFile
    -traceFile "~/test.trace.db" -javaClass "Test"
```

The generated file Test.java will contain the Java source code. The generated source code may be too large compile (the size of a Java method is limited). If this is the case, the source code needs to be split in multiple methods. The password is not listed in the trace file and therefore not included in the source code.

Using Other Logging APIs

By default, this database uses its own native 'trace' facility. This facility is called 'trace' and not 'log' within this database to avoid confusion with the transaction log. Trace messages can be written to both file and System.out . In most cases, this is sufficient, however sometimes it is better to use the same facility as the application, for example Log4j. To do that, this database support SLF4J.

SLF4J is a simple facade for various logging APIs and allows to plug in the desired implementation at deployment time. SLF4J supports implementations such as Logback, Log4j, Jakarta Commons Logging (JCL), Java logging, x4juli, and Simple Log.

To enable SLF4J, set the file trace level to 4 in the database URL:

```
jdbc:h2:~/test;TRACE_LEVEL_FILE=4
```

Changing the log mechanism is not possible after the database is open, that means executing the SQL statem SET TRACE_LEVEL_FILE 4 when the database is already open will not have the desired effect. To use SLF all required jar files need to be in the classpath. The logger name is h2database . If it does not work, check th file <database>.trace.db for error messages.

Read Only Databases

If the database files are read-only, then the database is read-only as well. It is not possible to create new table add or modify data in this database. Only SELECT and CALL statements are allowed. To create a read-only database, close the database. Then, make the database file read-only. When you open the database now, it is read-only. There are two ways an application can find out whether database is read-only: by calling Connection.isReadOnly() or by executing the SQL statement CALL READONLY() .

Using the Custom Access Mode r the database can also be opened in read-only mode, even if the database is not read only.

Read Only Databases in Zip or Jar File

To create a read-only database in a zip file, first create a regular persistent database, and then create a backu The database must not have pending changes, that means you need to close all connections to the database first. To speed up opening the read-only database and running queries, the database should be closed using SHUTDOWN DEFRAG . If you are using a database named test , an easy way to create a zip file is using th Backup tool. You can start the tool from the command line, or from within the H2 Console (Tools - Backup). Please note that the database must be closed when the backup is created. Therefore, the SQL statement BACKUP TO can not be used.

When the zip file is created, you can open the database in the zip file using the following database URL:

```
jdbc:h2:zip:~/data.zip!/test
```

Databases in zip files are read-only. The performance for some queries will be slower than when using a regula database, because random access in zip files is not supported (only streaming). How much this affects the performance depends on the queries and the data. The database is not read in memory; therefore large databases are supported as well. The same indexes are used as when using a regular database.

If the database is larger than a few megabytes, performance is much better if the database file is split into multiple smaller files, because random access in compressed files is not possible. See also the sample application ReadOnlyDatabaseInZip.

Opening a Corrupted Database

If a database cannot be opened because the boot info (the SQL script that is run at startup) is corrupted, then database can be opened by specifying a database event listener. The exceptions are logged, but opening the database will continue.

Computed Columns / Function Based Index

A computed column is a column whose value is calculated before storing. The formula is evaluated when the r is inserted, and re-evaluated every time the row is updated. One use case is to automatically update the last-modification time:

```
CREATE TABLE TEST(ID INT, NAME VARCHAR, LAST_MOD TIMESTAMP AS NOW());
```

Function indexes are not directly supported by this database, but they can be emulated by using computed columns. For example, if an index on the upper-case version of a column is required, create a computed colun with the upper-case version of the original column, and create an index for this column:

```
CREATE TABLE ADDRESS(
    ID INT PRIMARY KEY,
    NAME VARCHAR,
    UPPER_NAME VARCHAR AS UPPER(NAME)
);
CREATE INDEX IDX_U_NAME ON ADDRESS(UPPER_NAME);
```

When inserting data, it is not required (and not allowed) to specify a value for the upper-case version of the column, because the value is generated. But you can use the column when querying the table:

```
INSERT INTO ADDRESS(ID, NAME) VALUES(1, 'Miller');
SELECT * FROM ADDRESS WHERE UPPER_NAME='MILLER';
```

Multi-Dimensional Indexes

A tool is provided to execute efficient multi-dimension (spatial) range queries. This database does not support specialized spatial index (R-Tree or similar). Instead, the B-Tree index is used. For each record, the multi-dimensional key is converted (mapped) to a single dimensional (scalar) value. This value specifies the locatio on a space-filling curve.

Currently, Z-order (also called N-order or Morton-order) is used; Hilbert curve could also be used, but the implementation is more complex. The algorithm to convert the multi-dimensional value is called bit-interleavin The scalar value is indexed using a B-Tree index (usually using a computed column).

The method can result in a drastic performance improvement over just using an index on the first column. Depending on the data and number of dimensions, the improvement is usually higher than factor 5. The tool generates a SQL query from a specified multi-dimensional range. The method used is not database dependent and the tool can easily be ported to other databases. For an example how to use the tool, please have a look a the sample code provided in TestMultiDimension.java .

User-Defined Functions and Stored Procedures

In addition to the built-in functions, this database supports user-defined Java functions. In this database, Java functions can be used as stored procedures as well. A function must be declared (registered) before it can be used. A function can be defined using source code, or as a reference to a compiled class that is available in t classpath. By default, the function aliases are stored in the current schema.

Referencing a Compiled Method

When referencing a method, the class must already be compiled and included in the classpath where the database is running. Only static Java methods are supported; both the class and the method must be public. Example Java class:

```
package acme;
import java.math.*;
public class Function {
    public static boolean isPrime(int value) {
        return new BigInteger(String.valueOf(value)).isProbablePrime(100);
    }
}
```

The Java function must be registered in the database by calling CREATE ALIAS ... FOR :

```
CREATE ALIAS IS_PRIME FOR "acme.Function.isPrime";
```

For a complete sample application, see src/test/org/h2/samples/Function.java .

Declaring Functions as Source Code

When defining a function alias with source code, the database tries to compile the source code using the Sun Java compiler (the class com.sun.tools.javac.Main) if the tools.jar is in the classpath. If not, javac is run a separate process. Only the source code is stored in the database; the class is compiled each time the databa is re-opened. Source code is usually passed as dollar quoted text to avoid escaping problems, however single quotes can be used as well. Example:

```
CREATE ALIAS NEXT_PRIME AS $$
String nextPrime(String value) {
    return new BigInteger(value).nextProbablePrime().toString();
}
$$;
```

By default, the three packages java.util, java.math, java.sql are imported. The method name (nextPrime in t example above) is ignored. Method overloading is not supported when declaring functions as source code, tha means only one method may be declared for an alias. If different import statements are required, they must be declared at the beginning and separated with the tag @CODE :

```
CREATE ALIAS IP_ADDRESS AS $$
import java.net.*;
```

```
@CODE
String ipAddress(String host) throws Exception {
    return InetAddress.getByName(host).getHostAddress();
}
$$;
```

The following template is used to create a complete Java class:

```
package org.h2.dynamic;
< import statements before the tag @CODE; if not set:
import java.util.*;
import java.math.*;
import java.sql.*;
>
public class <aliasName> {
    public static <sourceCode>
}
```

Method Overloading

Multiple methods may be bound to a SQL function if the class is already compiled and included in the classpa Each Java method must have a different number of arguments. Method overloading is not supported when declaring functions as source code.

Function Data Type Mapping

Functions that accept non-nullable parameters such as `int` will not be called if one of those parameters is `NULL` . Instead, the result of the function is `NULL` . If the function should be called if a parameter is `NULL` , y need to use `java.lang.Integer` instead.

SQL types are mapped to Java classes and vice-versa as in the JDBC API. For details, see Data Types. Ther are a few special cases: `java.lang.Object` is mapped to `OTHER` (a serialized object). Therefore, `java.lang.Object` can not be used to match all SQL types (matching all SQL types is not supported). The sec special case is `Object[]` : arrays of any class are mapped to `ARRAY` . Objects of type `org.h2.value.Value` (tl internal value class) are passed through without conversion.

Functions That Require a Connection

If the first parameter of a Java function is a `java.sql.Connection` , then the connection to database is provided This connection does not need to be closed before returning. When calling the method from within the SQL statement, this connection parameter does not need to be (can not be) specified.

Functions Throwing an Exception

If a function throws an exception, then the current statement is rolled back and the exception is thrown to the application. SQLException are directly re-thrown to the calling application; all other exceptions are first convert to a SQLException.

Functions Returning a Result Set

Functions may returns a result set. Such a function can be called with the `CALL` statement:

```
public static ResultSet query(Connection conn, String sql) throws SQLException {
    return conn.createStatement().executeQuery(sql);
}

CREATE ALIAS QUERY FOR "org.h2.samples.Function.query";
CALL QUERY('SELECT * FROM TEST');
```

Using SimpleResultSet

A function can create a result set using the `SimpleResultSet` tool:

```
import org.h2.tools.SimpleResultSet;
…
public static ResultSet simpleResultSet() throws SQLException {
```

```
    SimpleResultSet rs = new SimpleResultSet();
    rs.addColumn("ID", Types.INTEGER, 10, 0);
    rs.addColumn("NAME", Types.VARCHAR, 255, 0);
    rs.addRow(0, "Hello");
    rs.addRow(1, "World");
    return rs;
}

CREATE ALIAS SIMPLE FOR "org.h2.samples.Function.simpleResultSet";
CALL SIMPLE();
```

Using a Function as a Table

A function that returns a result set can be used like a table. However, in this case the function is called at leas twice: first while parsing the statement to collect the column names (with parameters set to `null` where not known at compile time). And then, while executing the statement to get the data (maybe multiple times if this i join). If the function is called just to get the column list, the URL of the connection passed to the function is `jdbc:columnlist:connection` . Otherwise, the URL of the connection is `jdbc:default:connection` .

```
public static ResultSet getMatrix(Connection conn, Integer size)
        throws SQLException {
    SimpleResultSet rs = new SimpleResultSet();
    rs.addColumn("X", Types.INTEGER, 10, 0);
    rs.addColumn("Y", Types.INTEGER, 10, 0);
    String url = conn.getMetaData().getURL();
    if (url.equals("jdbc:columnlist:connection")) {
        return rs;
    }
    for (int s = size.intValue(), x = 0; x < s; x++) {
        for (int y = 0; y < s; y++) {
            rs.addRow(x, y);
        }
    }
    return rs;
}

CREATE ALIAS MATRIX FOR "org.h2.samples.Function.getMatrix";
SELECT * FROM MATRIX(4) ORDER BY X, Y;
```

Pluggable or User-Defined Tables

For situations where you need to expose other data-sources to the SQL engine as a table, there are "pluggable tables". For some examples, have a look at the code in `org.h2.test.db.TestTableEngines` .

In order to create your own TableEngine, you need to implement the `org.h2.api.TableEngine` interface e.g. something like this:

```
package acme;
public static class MyTableEngine implements org.h2.api.TableEngine {

    private static class MyTable extends org.h2.table.TableBase {
        .. rather a lot of code here...
    }

    public EndlessTable createTable(CreateTableData data) {
        return new EndlessTable(data);
    }
}
```

and then create the table from SQL like this:

```
CREATE TABLE TEST(ID INT, NAME VARCHAR)
    ENGINE "acme.MyTableEngine";
```

It is also possible to pass in parameters to the table engine, like so:

```
CREATE TABLE TEST(ID INT, NAME VARCHAR) ENGINE "acme.MyTableEngine" WITH "param1", "param2
```

In which case the parameters are passed down in the tableEngineParams field of the CreateTableData object.

Triggers

This database supports Java triggers that are called before or after a row is updated, inserted or deleted. Trigg can be used for complex consistency checks, or to update related data in the database. It is also possible to u triggers to simulate materialized views. For a complete sample application, see src/test/org/h2/samples/TriggerSample.java . A Java trigger must implement the interface org.h2.api.Trigger . The trigger class must be available in the classpath of the database engine (when using the server mode, it m be in the classpath of the server).

```
import org.h2.api.Trigger;
...
public class TriggerSample implements Trigger {

    public void init(Connection conn, String schemaName, String triggerName,
            String tableName, boolean before, int type) {
        // initialize the trigger object is necessary
    }

    public void fire(Connection conn,
            Object[] oldRow, Object[] newRow)
            throws SQLException {
        // the trigger is fired
    }

    public void close() {
        // the database is closed
    }

    public void remove() {
        // the trigger was dropped
    }

}
```

The connection can be used to query or update data in other tables. The trigger then needs to be defined in the database:

```
CREATE TRIGGER INV_INS AFTER INSERT ON INVOICE
    FOR EACH ROW CALL "org.h2.samples.TriggerSample"
```

The trigger can be used to veto a change by throwing a SQLException .

As an alternative to implementing the Trigger interface, an application can extend the abstract class org.h2.tools.TriggerAdapter . This will allows to use the ResultSet interface within trigger implementations. In this case, only the fire method needs to be implemented:

```
import org.h2.tools.TriggerAdapter;
...
public class TriggerSample implements TriggerAdapter {

    public void fire(Connection conn, ResultSet oldRow, ResultSet newRow)
            throws SQLException {
        // the trigger is fired
    }

}
```

Compacting a Database

Empty space in the database file re-used automatically. When closing the database, the database is automatically compacted for up to 200 milliseconds by default. To compact more, use the SQL statement SHUTDOWN COMPACT. However re-creating the database may further reduce the database size because thi will re-build the indexes. Here is a sample function to do this:

```
public static void compact(String dir, String dbName,
        String user, String password) throws Exception {
    String url = "jdbc:h2:" + dir + "/" + dbName;
    String file = "data/test.sql";
    Script.execute(url, user, password, file);
    DeleteDbFiles.execute(dir, dbName, true);
    RunScript.execute(url, user, password, file, null, false);
}
```

See also the sample application org.h2.samples.Compact . The commands SCRIPT / RUNSCRIPT can be used as well to create a backup of a database and re-build the database from the script.

Cache Settings

The database keeps most frequently used data in the main memory. The amount of memory used for caching can be changed using the setting CACHE_SIZE . This setting can be set in the database connection URL (jdbc:h2:~/test;CACHE_SIZE=131072), or it can be changed at runtime using SET CACHE_SIZE size . The size of the cache, as represented by CACHE_SIZE is measured in KB, with each KB being 1024 bytes. This setting has no effect for in-memory databases. For persistent databases, the setting is stored in the database and re-used when the database is opened the next time. However, when opening an existing database, the cac size is set to at most half the amount of memory available for the virtual machine (Runtime.getRuntime().maxMemory()), even if the cache size setting stored in the database is larger; however setting stored in the database is kept. Setting the cache size in the database URL or explicitly using SET CACHE_SIZE overrides this value (even if larger than the physical memory). To get the current used maximu cache size, use the query SELECT * FROM INFORMATION_SCHEMA.SETTINGS WHERE NAME = 'info.CACHE_MAX_SIZE'

An experimental scan-resistant cache algorithm "Two Queue" (2Q) is available. To enable it, append ;CACHE_TYPE=TQ to the database URL. The cache might not actually improve performance. If you plan to use it, please run your own test cases first.

Also included is an experimental second level soft reference cache. Rows in this cache are only garbage collected on low memory. By default the second level cache is disabled. To enable it, use the prefix SOFT_ . Example: jdbc:h2:~/test;CACHE_TYPE=SOFT_LRU . The cache might not actually improve performance. If plan to use it, please run your own test cases first.

To get information about page reads and writes, and the current caching algorithm in use, call SELECT * FRO INFORMATION_SCHEMA.SETTINGS . The number of pages read / written is listed.